ترتيب الإدخال
(Insertion Sort)

كيف نرتب مصفوفة خطوة بخطوة؟

إعداد: الأستاذ محمود اغبارية

↔ تذكير: الإزاحة (Shift Right)

في البرمجة لا "ننقل" القيمة فنترك مكانها فارغاً، بل **ننسخها (Copy)** فوق الخانة المجاورة.

انتهت الإزاحة! الفهرس 1 يحتوي على قيمة قديمة (20) يمكن الكتابة فوقها بأمان.

10	20	20	30	40
0	1	2	3	4

السابق 5 / 5 التالي

```
// إزاحة العناصر من الفهرس 1 حتى 3 خطوة لليمين
for (int j = 3; j ≥ 1; j--) {
    arr[j + 1] = arr[j];
}
```

القاعدة الذهبية: عند الإزاحة لليمين، **نبدأ دائماً من الأخير** لتجنب طمس البيانات.

🧩 مسألة: ترتيب العنصر الأخير

معطاة مصفوفة أعداد صحيحة `arr`، جميع عناصرها **مرتبة تصاعدياً** ما عدا **العنصر الأخير** الذي أُضيف للتو وقد يكون في المكان الخاطئ.

اكتب عملية خارجية `SortLastItem` تتلقى هذه المصفوفة وتُدرج العنصر الأخير في مكانه الصحيح.

مثال:

قبل الاستدعاء

3 | 7 | 10 | 15 | 6

→

بعد الاستدعاء

3 | 6 | 7 | 10 | 15

تلميح – خطة الحل:

1. احفظ قيمة العنصر الأخير في متغير مساعد (`tmp`).
2. ابدأ من العنصر الذي قبل الأخير وتحرك للخلف.
3. طالما العنصر الحالي أكبر من `tmp`، انسخه خطوة لليمين.
4. ضع `tmp` في المكان المناسب.

الحل: SortLastItem

```
public static void SortLastItem(int[] arr)
{
    int tmp = arr[arr.Length - 1]; // احفظ العنصر الأخير
    int idx = arr.Length - 2;      // ابدأ من قبل الأخير

    while (idx ≥ 0 && arr[idx] > tmp)
    {
        arr[idx + 1] = arr[idx]; // انسخ العنصر خطوة لليمين
        idx--;
```

ماذا يفعل الكود؟

يحفظ العنصر الأخير، ثم يُزيح العناصر الأكبر منه خطوة لليمين (من الأخير باتجاه البداية)، حتى يجد مكانه المناسب ويكتبه هناك.

لاحظ: هذا الكود يحل مشكلة **عنصر واحد** غير مرتب في نهاية المصفوفة. ماذا لو أردنا ترتيب **مصفوفة كاملة** بنفس الفكرة؟ هذا بالضبط ما تفعله **Insertion Sort**!

خوارزمية Insertion Sort: الفكرة العامة

نطبق نفس فكرة `SortLastItem` على كل عنصر في المصفوفة، واحداً تلو الآخر:

- نعتبر **العنصر الأول** (الفهرس 0) مرتباً بحد ذاته.
- نبدأ حلقة تمر على باقي العناصر ابتداءً من **الفهرس 1**.
- في كل دورة، **ننسخ** العنصر الحالي إلى متغير جانبي يُسمى **key** (لحمايته من الطمس).
- نقارن الـ **key** بالعناصر التي تسبقه **من اليمين إلى اليسار**.
- طالما العنصر السابق **أكبر** من الـ **key**، ننسخه خطوة لليمين.
- عندما نجد مكانًا مناسبًا، **نكتب** الـ **key** فيه.

> **المفتاح:** في كل دورة، القسم الأيسر (من الفهرس 0 حتى i) يكون **مرتباً دائماً**، ويكبر تدريجياً حتى تمتد إلى كامل المصفوفة.

محاكاة بصرية: Insertion Sort على المصفوفة [7, 3, 5, 2, 6]

انتهت الخوارزمية! المصفوفة مرتبة بالكامل 🎉

السابق 26 / 26 التالي

</> كود C# الكامل

```
public static void InsertionSort(int[] arr)
{
    for (int i = 1; i < arr.Length; i++) {
        int key = arr[i]; // ننسخ العنصر للمتغير لكي لا يُطمس
        int j = i - 1;    // j يؤشر على العناصر المرتبة التي تسبقه

        // key طالما لم نصل للبداية والعنصر السابق أكبر من الـ
        while (j ≥ 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // ننسخ العنصر لليمين
            j--;                 // نتراجع خطوة للخلف
        }
```

قارن مع SortLastItem: كود Insertion Sort هو نفس فكرة `SortLastItem` تماماً، لكن داخل حلقة `for` تمر على كل عناصر المصفوفة من الفهرس 1 وحتى النهاية.

? تمرين 1: متابعة (מעקב)

معطاة المصفوفة التالية:

2	6	4	8

سؤال: كيف ستبدو المصفوفة `i = 2` بعد انتهاء الدورة التي يكون فيها تماماً؟

فكر في الإجابة، ثم انتقل للشريحة التالية.

إجابة تمرين 1

الإجابة: { 4, 6, 8, 2 }

الشرح خطوة بخطوة:

- **قبل البداية (i = 0):** { 8, 4, 6, 2 }
- **عندما i = 1 (الـ key = 4):** 8 أكبر منه ← ننسخ 8 لليمين ← نكتب 4.
  تصبح: { 4, 8, 6, 2 }
- **عندما i = 2 (الـ key = 6):** 8 أكبر ← ننسخ ← 4 أصغر ← نتوقف ← نكتب 6.
  تصبح: { 4, 6, 8, 2 }

تمرين 2: شرط التوقف

في حلقة الـ `while` نكتب الشرط المركب:

```
while (j >= 0 && arr[j] > key)
```

سؤال: ماذا يحدث إذا نسينا الشرط `j ≥ 0` وتركنا `arr[j] > key` فقط؟

تلميح: ماذا يحدث عندما يكون الـ key هو أصغر رقم في المصفوفة؟

إجابة تمرين 2

خطأ وقت التشغيل! (IndexOutOfRangeException) 💥

التفسير:

إذا كان الـ key هو أصغر عنصر، فإن حلقة الـ while لن تجد عنصراً أصغر منه أبداً. ستستمر في تنقيص `j` حتى تصبح `-1`.

عندها سيحاول الحاسوب الوصول إلى `arr[-1]` وهو فهرس غير موجود ← البرنامج ينهار.

> الشرط `j ⩾ 0` هو "صمام الأمان" الذي يوقف الحلقة عند الوصول لبداية المصفوفة.

تمرين 3: كفاءة الخوارزمية

لنفترض أننا أعطينا الكود مصفوفة **مرتبة مسبقاً**، مثل:

1	2	3	4	5

سؤال: كم مرة سيتم تنفيذ الأوامر داخل حلقة الـ while؟ ولماذا؟

إجابة تمرين 3

لن يتم الدخول لحلقة الـ while أبداً! (0 مرات)

التفسير:

في كل دورة `i`، يقارن البرنامج الـ key بالعنصر الذي يسبقه مباشرة.

بما أن المصفوفة مرتبة، العنصر السابق سيكون دائماً **أصغر** من الـ key، فالشرط `arr[j] > key` يُعطي **false** مباشرة ولا ندخل الـ while.

معلومة: المصفوفة المرتبة مسبقاً هي "أفضل حالة" (Best Case) لخوارزمية Insertion Sort، وتكون كفاءتها $O(n)$.

🏆 تمرين 4: تحدي البرمجة

الكود الأساسي يرتب المصفوفة **تصاعدياً** (من الأصغر للأكبر).

سؤال: ما هو التعديل الوحيد في شرط الـ while لترتيب المصفوفة تنازلياً (من الأكبر للأصغر)؟

```
while (j ≥ 0 && arr[j] ??? key)
```

إجابة تمرين 4

نغير > إلى <

```
while (j ≥ 0 && arr[j] < key)
```

التفسير:

في الترتيب التصاعدي: نزيح العناصر **الأكبر** من الـ key لليمين لإفساح مكان لعنصر أصغر.

في الترتيب التنازلي: نريد أن تأتي الأعداد الكبيرة أولاً، لذلك نزيح العناصر **الأصغر** لليمين لإفساح مكان لعنصر أكبر.

نهاية الدرس

كل التوفيق والنجاح!

تذكر دائماً: البرمجة تحتاج لورقة وقلم وتتبع الكود خطوة بخطوة.

أرجو لكم عملاً ممتعاً! / الأستاذ محمود اغبارية